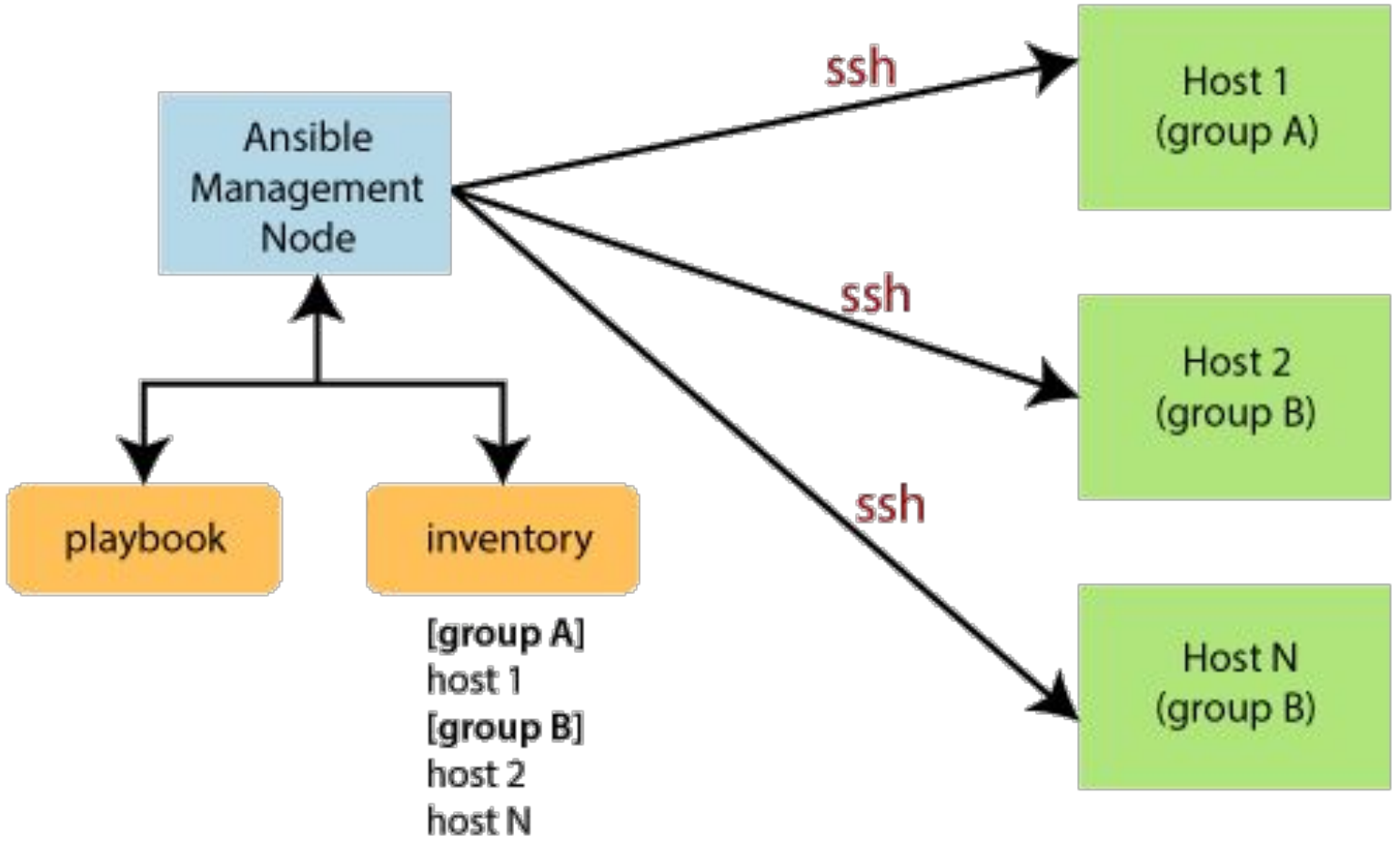
DevOps

Ansible

What is Ansible?

- Ansible is an open-source automation tool used for configuring, managing, and deploying software applications on various servers and devices. It uses a simple and easy-to-understand syntax called YAML to define automation tasks, and it can automate tasks across a wide range of systems, including Linux, Unix, and Windows systems.
- What can Ansible do?
  - Configuration of servers
  - Application deployment
  - Continuous testing of already install application
  - Provisioning
  - Orchestration
  - Automation of tasks

How does Ansible work?



Ansible Installation and Configuration

- On Ubuntu

```
$ sudo apt update
$ sudo apt install software-properties-common
$ sudo apt-add-repository --yes --update ppa:ansible/ansible
$ sudo apt install ansible
$ ansible --version
```

- With Python

```
$ sudo apt install python3-pip # Install Package Manager for Python 3
$ sudo pip3 install ansible
$ ansible --version
```

Ansible Installation and Configuration

```
$ ssh-keygen # Generate new ssh key without password
```

- Copy ssh-key public (id_rsa.pub) to ~/.ssh/authorized_keys in all servers (include ownself)
- Try to connect all servers via ssh for testing

```
ubuntu@ip-172-31-23-222:~/.ssh$ ls
authorized_keys  id_rsa  id_rsa.pub  known_hosts  known_hosts.old
ubuntu@ip-172-31-23-222:~/.ssh$ cat id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDBQF/dhwlysezaTdS3re80k8lK26fsGxAx+Ey866dtAjMA1iDeyu2chC8GSsPdZtrnMmpmT76o5Ba/GLTe
3QL80rn2h/2uXVCfwXSUo6YghcsnktQor2cn17hBMVdadMgymnCf5FJhC1FNFYYiqEOFzqQOuizFI7+JN0WDoDs483izLxW4VM8DIQeS9UUnnlCklhMsf+HO
h7VMpQ5jYiyc7kE+3HgSK7XGBSduL0LKBAGKiJI5EYe+a3PelQAG9Qi4g5N0NYFAL5T1j3wn3ZDl3cdIfrIbmo57FwjcIWfDhBQrvMUhTS3EBSZITc7iRLSH
VhhWa+0ZtKA5tD3nooXbtL0y3j4Is/4ENMgSgEMTf8nZIJ/jIqPl13G5etCN4rJXcVnq9faQkZQIcf6smkJOc5hfjbKUPvX5Ow5LFt1QUAxdWz9dKtezcaTj
eIF3I9SuViJhrYyYscWoFsdxuoZbw7vPzZolfy2nWPZ2tHF64z8Q4RCCpOQ3xvircLgWvvk= ubuntu@ip-172-31-23-222
```

```
ubuntu@ip-172-31-29-236:~/.ssh$ ls
authorized_keys
ubuntu@ip-172-31-29-236:~/.ssh$ cat authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQCtrgWUo3tuAUEAEf3tlkzRxMvT9ZCGQRaLn9A54T5xU0+TsxrJG0+gq91KgZj9/lezs9zH7xJj6k1sOJj1
WphzEWOhtx4N5ICosab/YhVpkn0JImlfnvKpTlXZy0UfdabM9+zboogINDlc/YVRNnUJci2HffwH9tLdxo6YnC6StL9QVS43Z9cmWpz9PKhYbQpxIdZBrHDv
1h44Hk8IbeE/zkzQYbq0VXp1E+SVLY2y/BoxgAIofXOu6LqXXKVonWxMFeTRoScRPr7d5Aoyz7o/FeIqC3dAHJBJkljzTsPQ5aiQANPPbXQamvraoKkp+cAX
cj8eBySZ6w7jLKrllNPR aws-v3
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDBQF/dhwlysezaTdS3re80k8lK26fsGxAx+Ey866dtAjMA1iDeyu2chC8GSsPdZtrnMmpmT76o5Ba/GLTe
3QL80rn2h/2uXVCfwXSUo6YghcsnktQor2cn17hBMVdadMgymnCf5FJhC1FNFYYiqEOFzqQOuizFI7+JN0WDoDs483izLxW4VM8DIQeS9UUnnlCklhMsf+HO
h7VMpQ5jYiyc7kE+3HgSK7XGBSduL0LKBAGKiJI5EYe+a3PelQAG9Qi4g5N0NYFAL5T1j3wn3ZDl3cdIfrIbmo57FwjcIWfDhBQrvMUhTS3EBSZITc7iRLSH
VhhWa+0ZtKA5tD3nooXbtL0y3j4Is/4ENMgSgEMTf8nZIJ/jIqPl13G5etCN4rJXcVnq9faQkZQIcf6smkJOc5hfjbKUPvX5Ow5LFt1QUAxdWz9dKtezcaTj
eIF3I9SuViJhrYyYscWoFsdxuoZbw7vPzZolfy2nWPZ2tHF64z8Q4RCCpOQ3xvircLgWvvk= ubuntu@ip-172-31-23-222
```

Inventory

- Ansible works against multiple managed hosts in your infrastructure at the same time, using a list or group of lists is known as the inventory. Once an inventory is defined, you use patterns to select the hosts or groups you want to run against to Ansible. The default location for inventory is a file called **/etc/ansible/hosts**. You can also specify a different inventory file at the command line using the -i <path> option.

```
ubuntu@ip-172-31-23-222:~/ansible-folder$ ls
inventory1
ubuntu@ip-172-31-23-222:~/ansible-folder$ cat inventory1
[master]
172.31.23.222

[workers]
node2 ansible_host=172.31.29.236
node3 ansible_host=172.31.25.165

[all:children]
master
workers

[vars]
172.31.23.222
node2
```

```
ubuntu@ip-172-31-23-222:~/ansible-folder$ ansible -i inventory1 all -m ping
node2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
node3 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
172.31.23.222 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Module

- Ansible modules are discrete units of code which can be used from the command line or in a playbook task. They are the building blocks of Ansible that perform specific tasks, such as installing software, copying files, or starting services. Ansible has a large library of built-in modules and also supports custom modules.
- All Ansible Modules: https://docs.ansible.com/ansible/2.9/modules/list_of_all_modules.html
  - Module with Ad-hoc command

```
ubuntu@ip-172-31-23-222:~/ansible-folder$ ls
hello.txt  inventory1
ubuntu@ip-172-31-23-222:~/ansible-folder$ ansible -i inventory1 all -m copy -a "src=./hello.txt dest=/home/$USER"
node2 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": true,
    "checksum": "22596363b3de40b06f981fb85d82312e8c0ed511",
    "dest": "/home/ubuntu/hello.txt",
    "gid": 1000,
    "group": "ubuntu",
    "md5sum": "6f5902ac237024bdd0c176cb93063dc4",
    "mode": "0664",
    "owner": "ubuntu",
    "size": 12,
    "src": "/home/ubuntu/.ansible/tmp/ansible-tmp-1678230429.251697-4664-261827816636675/source",
    "state": "file",
    "uid": 1000
}
node3 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
```

Module

- Module with Ansible Playbook

```
ubuntu@ip-172-31-23-222:~/ansible-folder$ cat copy-playbook.yml
- name: Copy file to all servers
  hosts: all
  tasks:
    - name: Copy file
      copy:
        src: ./hello.txt
        dest: /home/$USER
        mode: u=rw,g=r,o-rwx
        #mode: "0640"
ubuntu@ip-172-31-23-222:~/ansible-folder$ ansible-playbook -i inventory1 copy-playbook.yml

PLAY [Copy file to all servers] ************************************************

TASK [Gathering Facts] *********************************************************
ok: [node2]
ok: [node3]
ok: [172.31.23.222]

TASK [Copy file] ***************************************************************
changed: [node3]
changed: [node2]
changed: [172.31.23.222]

PLAY RECAP *********************************************************************
172.31.23.222              : ok=2    changed=1    unreachable=0    failed=0    skipped=0
node2                      : ok=2    changed=1    unreachable=0    failed=0    skipped=0
node3                      : ok=2    changed=1    unreachable=0    failed=0    skipped=0
```

Ad-hoc Command

- Ansible ad-hoc commands are used for running quick, one-time tasks on remote hosts without the need for writing playbooks. Ad-hoc commands are ideal for tasks that need to be executed on a small number of hosts, such as running a single command, gathering information or installing a package.

```
$ ansible -i inventory all -m command -a "uptime"
$ ansible -i inventory workers -m apt -a "name=nginx state=present"
$ ansible -i inventory master -m shell -a "df -h"
$ ansible -i inventory vars -m shell -a "ls -l /var/www/html"
$ ansible all -i 172.31.20.111, -m file -a "path=/remote/path/to/file.txt state=touch"
$ ansible all -i example.com, -m copy -a "src=/local/path/to/script.sh
dest=/remote/path/to/script.sh mode=0755" -m shell -a "/remote/path/to/script.sh"
```

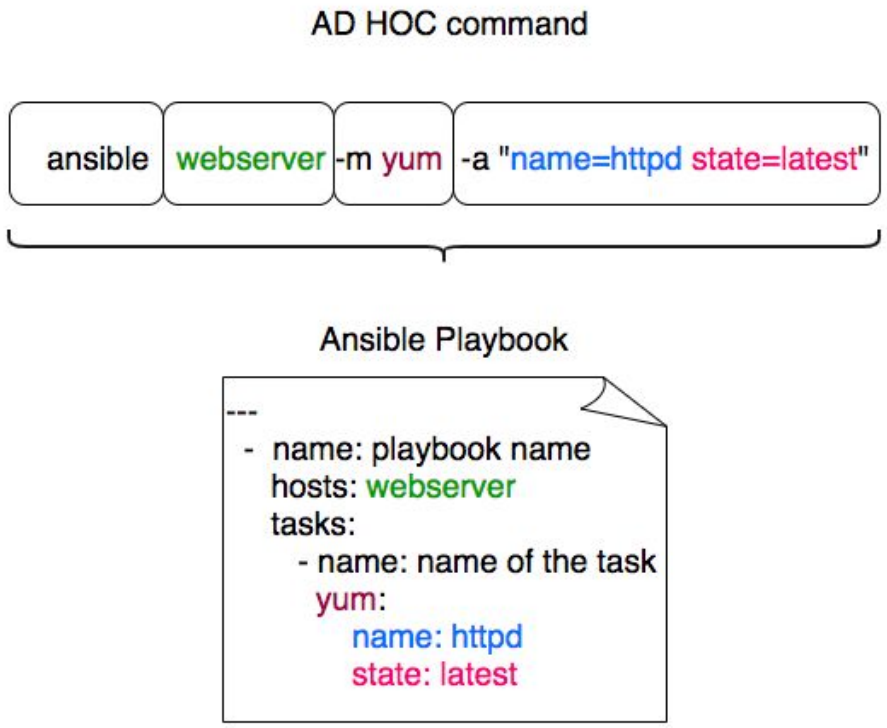
Ansible Common Options

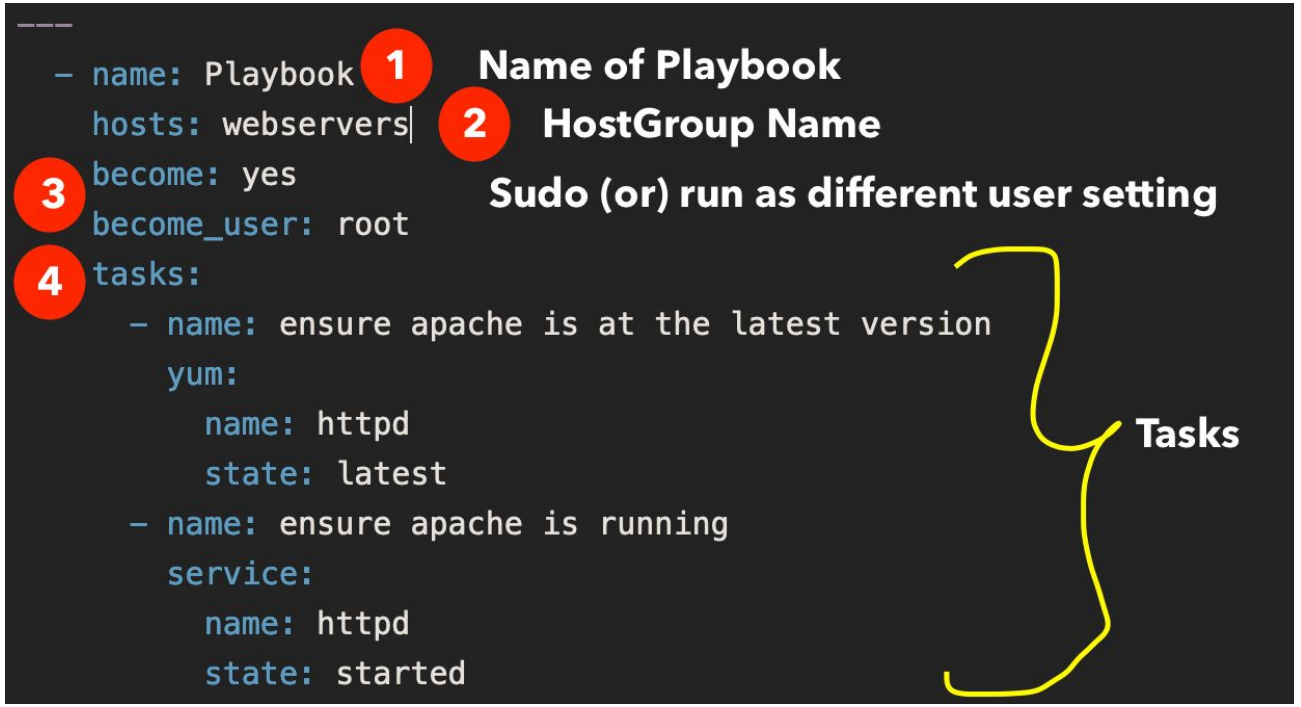
```
$ ansible -i inventory workers -u dara -b -K -f 3 -m apt -a "name=nginx state=present"
```

- `-i or --inventory:` The inventory file contains the list of hosts that Ansible will manage.
- `-u or --user:` The username to use when connecting to the target hosts.
- `--become-user:` The --become-user option specifies the user account that should be used for privilege escalation. By default, Ansible will use “sudo” if not will use “su” the root user account for privilege escalation.
- `-b or --become:` Executes with elevated privileges (sudo) no password implied.
- `--become-method:` privilege escalation method to use (default=sudo), valid choices: [sudo | su | pbrun | pfexec | doas | dzdo | ksu | runas | machinectl]
- `-K or --ask-become-pass:` Ask for privilege escalation password; does not imply become will be used. Note that this password will be used for all hosts.
- `-k or --ask-pass:` Ask for connection password and need sshpass.
- `-f or --folks:` Specify number of parallel processes to use (default=5).
- `-m or --module-name:` Name of the action to execute (default=command).
- `-C or --check:` Don’t make any changes; instead, try to predict some of the changes that may occur.
- `-a or --args:` The action’s options in space separated k=v format: -a ‘opt1=val1 opt2=val2’ or a json string: -a ‘{“opt1”: “val1”, “opt2”: “val2”}’.

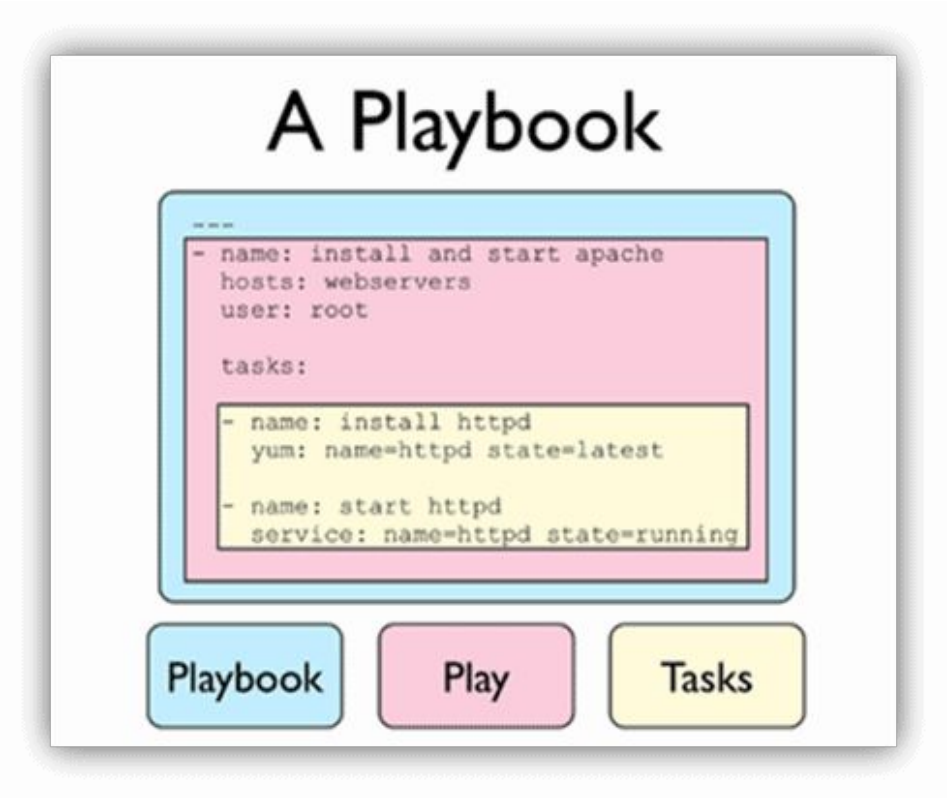
Ansible Playbook

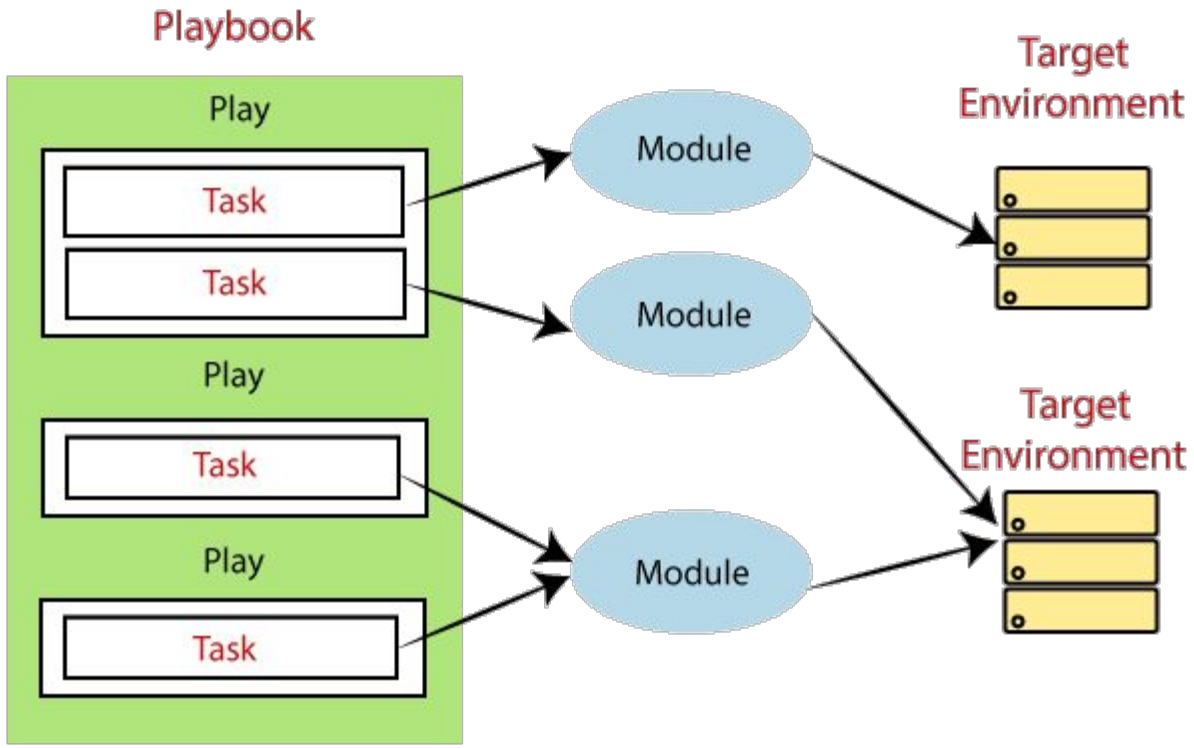
- Ansible playbook is a file where users write Ansible code, an organized collection of scripts defining the work of a server configuration. It contains a list of tasks of each play in an order they should get executed against a set of hosts or a single host based on the configuration specified. Playbooks are written in YAML, in an easy human-readable syntax





Ansible Playbook





Ansible Playbook

- Create file: my-play.yml, Run: **ansible-playbook -i inventory my-play.yml**

```yaml
---
- name: Install Nginx
  hosts: master
  become: true
  tasks:
   - name: Update apt cache
     apt:
       update_cache: yes

   - name: Install Nginx
     apt:
       name: nginx
       state: present
```

```yaml
- name: Create user
  hosts: workers
  become: true
  vars:
    user_name: jdoe
    user_password: mypassword
  tasks:
   - name: Create user
     user:
       name: "{{ user_name }}"
       password: "{{ user_password | password_hash('sha512') }}"
       shell: /bin/bash
       createhome: yes
       groups: sudo
```

Ansible Playbook

- Create file: compose-play.yml, Run: **ansible-playbook -i inventory compose-play.yml**

```
---
- name: Deploy application with Docker Compose
  hosts: workers
  become: true
  tasks:
   - name: Install Docker Compose
     apt:
       name: docker-compose
       state: present

   - name: Copy Docker Compose file
     copy:
       src: docker-compose.yml
       dest: /opt/docker-compose.yml

   - name: Start Docker Compose
     command: docker-compose up -d
     args:
       chdir: /opt
```

```
#docker-compose.yml
version: '3.8'
services:
 nginx:
   image: nginx
   ports:
     - "82:80"
```

Ansible Playbook

- Create file: compose-play.yml, Run: **ansible-playbook -i inventory compose-play.yml**

```yaml
---
- name: Deploy application with Docker Compose
  hosts: master
  become: true
  tasks:
   - name: Install Docker Compose
     apt:
       name: docker-compose
       state: present

   - name: Copy Docker Compose file
     copy:
       src: docker-compose.yml
       dest: /opt/docker-compose.yml

   - name: Start Docker Compose
     docker_compose:
       project_src: /opt
       project_name: myapp
       state: present
```

```yaml
#docker-compose.yml
version: '3.8'
services:
 nginx:
   image: nginx
   ports:
     - "82:80"
```

Ansible Playbook

- Create file: compose-play.yml, Run: **ansible-playbook -i inventory compose-play.yml**

```
---
- name: Deploy application with Docker Compose
  hosts: workers
  become: true
  tasks:
   - name: Install Docker Compose
     apt:
       name: docker-compose
       state: present

   - name: Start Docker Compose
     docker_compose:
       project_name: myapp
       definition:
         version: '3.8'
         services:
           nginx:
             image: nginx
             ports:
               - "82:80"
```

Variable

- In playbooks, the variable is very similar to using the variables in a programming language. It helps you to assign a value to a variable and use it anywhere in the playbook. You can put the conditions around the value of the variables and use them in the playbook accordingly.
- There are 3 main types of variables:
  - Playbook Variables
  - Inventory Variables
  - Special Variables

Playbook Variables

- Variable Priorities: “--extra-vars” > Playbook Variables > Inventory Variables

```yaml
# Create a file: vars/secrets.yml
- name: Create user
 hosts: workers
 become: true
 vars:
   user_name: jdoe
   user_password: mypassword
 vars_files:
   - vars/secrets.yml
 tasks:
   - name: Create user
     user:
       name:  "{{ user_name }}"
       password: " {{ user.pass | password_hash('sha512') }} "
       shell: /bin/bash
       createhome: yes
       groups: sudo
```

```yaml
#var/secrets.yml
user:
 name: dara
 pass: abc123
```

- $ ansible-playbook -i inventory compose-play.yml --extra-vars “user_name=dara user_password=123”
- Use “--extra-vars” to override the value of variables that have the same name.

Inventory Variables

- All Variables are defined which are available in your playbook and belong to specific host only.

```
[master]
172.31.23.222

[workers]
node2 ansible_host=172.31.29.236
node3 ansible_host=172.31.25.165

[all:children]
master
workers

[vars]
172.31.23.222
node2
```

```
localhost ansible_connection=local


[dev]
myserver.com ansible_user=dara ansible_host=10.1.1.1


[uat]
myserver.com ansible_user=dara ansible_host=10.2.2.2
```

Inventory Variables

- We can use **group_vars** and **host_vars** to store files that contain any variables. File names must match the group and host name and are stored in directories in the same path as the hosts file.

```yaml
- name: Create user
  hosts: all
  become: true
  tasks:
   - name: Create user
     user:
       name:  "{{ user.name }}"
       password: "{{ user.pass | password_hash('sha512') }}"
       shell: /bin/bash
       createhome: yes
       groups: sudo
```

```yaml
#host_vars/node2.yml
user:
 name: dara
 pass: dara123
```

```yaml
#host_vars/node3.yml
user:
 name: nita
 pass: nita123
```

```yaml
#group_vars/all.yml
user:
 name: istad
 pass: istad123
```

Special Variables

- Ansible defined variables, cannot be set by user.
- Resource: https://docs.ansible.com/ansible/latest/reference_appendices/special_variables.html
  - **inventory_hostname**: The inventory name for the current host being iterated over in the play.
  - **inventory_file**: The file name of the inventory source in which the inventory_hostname was first defined.
  - **group_names**: List of groups the current host is part of
  - **groups**: A dictionary/map with all the groups in inventory and each group has the list of hosts that belong to it.
  - **hostvars**: A dictionary/map with all the hosts in inventory and variables assigned to them.

Thank you

Begin with the theory, Start with the practical